MySQL Window Functions

1. What is a Window Function?

A Window Function performs a calculation on a group of related rows without combining those rows into one row.

Simple meaning

> Window Function = Calculate something across multiple rows, but keep every original row.

For example, suppose we have:

idnamedepartmentsalary1RahulIT500002AmanIT600003PriyaHR450004NehaHR55000

If we use `GROUP BY department`, we might get only one row per department.

But with a window function, we can calculate the department's average salary while still showing every employee.

2. Basic Syntax

```
function_name() OVER (
    PARTITION BY column
    ORDER BY column
);
```

There are three important parts:

`function_name()`

The calculation we want to perform.

Examples:

```
SUM()
AVG()
COUNT()
MAX()
MIN()
```

```
ROW_NUMBER()
RANK()
DENSE_RANK()
```

`OVER()`

Tells SQL:

> "This is a window function."

`PARTITION BY`

Divides the rows into groups.

`ORDER BY`

Decides the order in which the window calculation works.

3. Window Function vs GROUP BY

This is very important for exams and interviews.

GROUP BY

```
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
```

Output:

departmentavg_salaryIT55000HR50000

The individual employees are gone.

Window Function

```
SELECT
    name,
    department,
    salary,
    AVG(salary) OVER(PARTITION BY department) AS avg_salary
FROM employees;
```

Output:

namedepartmentsalaryavg_salaryRahulIT5000055000AmanIT6000055000PriyaHR4500050000NehaHR5500050000

Every employee remains visible.

4. PARTITION BY

`PARTITION BY` creates separate groups for the window calculation.

Example:

```
AVG(salary) OVER(PARTITION BY department)
```

It means:

> Calculate the average salary separately for each department.

So:

```
IT
 ├── Rahul  50000
 └── Aman   60000
       ↓
    Average = 55000

HR
 ├── Priya  45000
 └── Neha   55000
       ↓
    Average = 50000
```

5. SUM() with Window Function

Suppose:

namedepartmentsalaryRahulIT50000AmanIT60000PriyaHR45000NehaHR55000

Query:

```
SELECT
    name,
    department,
    salary,
    SUM(salary) OVER(PARTITION BY department) AS total_salary
FROM employees;
```

Output:

namedepartmentsalarytotal_salaryRahulIT50000110000AmanIT60000110000PriyaHR45000100000NehaHR55000100000

6. COUNT() with Window Function

```
SELECT
    name,
    department,
    COUNT(*) OVER(PARTITION BY department) AS employee_count
FROM employees;
```

Output:

namedepartmentemployee_countRahulIT2AmanIT2PriyaHR2NehaHR2

It counts employees inside each department.

7. MAX() and MIN()

Maximum salary in each department

```
SELECT
    name,
    department,
    salary,
    MAX(salary) OVER(PARTITION BY department) AS max_salary
FROM employees;
```

Minimum salary

```
SELECT
    name,
    department,
    salary,
    MIN(salary) OVER(PARTITION BY department) AS min_salary
FROM employees;
```

8. ROW_NUMBER()

`ROW_NUMBER()` gives a unique number to every row.

Example:

```
SELECT
    name,
    salary,
    ROW_NUMBER() OVER(ORDER BY salary DESC) AS row_num
FROM employees;
```

Output:

namesalaryrow_numAman600001Neha550002Rahul500003Priya450004

Important

Highest salary gets:

```
1
```

Second highest:

```
2
```

and so on.

9. ROW_NUMBER() with PARTITION BY

This is very important.

```
SELECT
    name,
    department,
    salary,
    ROW_NUMBER() OVER(
        PARTITION BY department
        ORDER BY salary DESC
    ) AS row_num
FROM employees;
```

Output:

namedepartmentsalaryrow_numAmanIT600001RahulIT500002NehaHR550001PriyaHR450002

Notice:

IT starts from `1`.

HR also starts from `1`.

Because `PARTITION BY department` creates separate groups.

10. RANK()

`RANK()` gives the same rank when values are equal.

Example:

namesalaryAman60000Rahul60000Neha50000Priya40000

```
SELECT
    name,
    salary,
    RANK() OVER(ORDER BY salary DESC) AS salary_rank
FROM employees;
```

Output:

namesalarysalary_rankAman600001Rahul600001Neha500003Priya400004

Notice:

```
1
1
3
4
```

Rank `2` is skipped.

11. DENSE_RANK()

`DENSE_RANK()` also gives the same rank for equal values.

But it does not skip the next rank.

```
SELECT
    name,
    salary,
    DENSE_RANK() OVER(ORDER BY salary DESC) AS salary_rank
FROM employees;
```

Output:

namesalarysalary_rankAman600001Rahul600001Neha500002Priya400003

Easy difference

Suppose salaries are:

```
60000
60000
50000
40000
```

`ROW_NUMBER()`:

```
1
2
3
4
```

`RANK()`:

```
1
1
3
4
```

`DENSE_RANK()`:

```
1
1
2
3
```

12. LAG()

`LAG()` allows us to access the previous row.

Example:

monthsalesJan10000Feb15000Mar12000

Query:

```
SELECT
    month,
    sales,
    LAG(sales) OVER(ORDER BY month) AS previous_sales
FROM sales;
```

Output:

monthsalesprevious_salesJan10000NULLFeb1500010000Mar1200015000

So:

```
Jan → no previous row → NULL
Feb → previous = Jan
Mar → previous = Feb
```

13. LEAD()

`LEAD()` is opposite of `LAG()`.

It gives us the next row.

```
SELECT
    month,
    sales,
    LEAD(sales) OVER(ORDER BY month) AS next_sales
FROM sales;
```

Output:

monthsalesnext_salesJan1000015000Feb1500012000Mar12000NULL

Remember:

```
LAG  → Previous row
LEAD → Next row
```

14. Calculating Difference Between Rows

This is a very useful real-world example.

```
SELECT
    month,
    sales,
    sales - LAG(sales) OVER(ORDER BY month) AS difference
FROM sales;
```

Output:

monthsalesdifferenceJan10000NULLFeb150005000Mar12000-3000

Meaning:

```
Feb:
15000 - 10000 = 5000

Mar:
```

```
12000 - 15000 = -3000
```

15. Running Total

A running total means:

> Keep adding the current value to all previous values.

Example:

monthsalesJan10000Feb15000Mar12000

Query:

```
SELECT
    month,
    sales,
    SUM(sales) OVER(
        ORDER BY month
    ) AS running_total
FROM sales;
```

Output:

monthsalesrunning_totalJan1000010000Feb1500025000Mar1200037000

Calculation:

```
Jan = 10000

Feb = 10000 + 15000
    = 25000

Mar = 10000 + 15000 + 12000
    = 37000
```

16. Running Total for Each Department

We can combine `PARTITION BY` and `ORDER BY`.

```
SELECT
    name,
```

```
    department,
    salary,
    SUM(salary) OVER(
        PARTITION BY department
        ORDER BY salary
    ) AS running_total
FROM employees;
```

The running total starts separately for every department.

17. Important Window Functions

For beginner/interview/exam preparation, remember these:

FunctionMeaning`SUM()`Total`AVG()`Average`COUNT()`Count`MAX()`Maximum`MIN()`Minimum`ROW_NUMBER()`Unique row number`RANK()`Ranking with gaps`DENSE_RANK()`Ranking without gaps`LAG()`Previous row`LEAD()`Next row

18. Very Important Syntax

Without partition

```
ROW_NUMBER() OVER(ORDER BY salary DESC)
```

All rows are considered together.

With partition

```
ROW_NUMBER() OVER(
    PARTITION BY department
    ORDER BY salary DESC
)
```

Rows are divided department-wise first.

19. Window Function Does NOT Reduce Rows

This is the easiest way to remember it.

GROUP BY

```
10 employees
      ↓
GROUP BY
      ↓
2 rows
```

Window Function

```
10 employees
      ↓
Window Function
      ↓
10 rows
```

So:

> GROUP BY reduces rows. Window functions keep the rows.

20. Example: Find Highest-Paid Employee in Each Department

First assign ranking:

```
SELECT
    name,
    department,
    salary,
    RANK() OVER(
        PARTITION BY department
        ORDER BY salary DESC
    ) AS rnk
FROM employees;
```

Then use a subquery:

```
SELECT *
FROM (
    SELECT
        name,
        department,
        salary,
        RANK() OVER(
            PARTITION BY department
            ORDER BY salary DESC
        ) AS rnk
    FROM employees
) AS x
WHERE rnk = 1;
```

This gives the employee(s) having the highest salary in each department.

21. Window Function Execution Idea

Suppose:

```
SELECT
    name,
    salary,
    AVG(salary) OVER() AS avg_salary
FROM employees;
```

Think of SQL as:

```
Employees
   ↓
Look at all rows
   ↓
Calculate average
   ↓
Attach average to every row
```

So if average salary is `50,000`:

```
Rahul    50000    50000
Aman     60000    50000
Priya    45000    50000
Neha     55000    50000
```

The average is repeated because the original rows are preserved.

22. Exam Points to Remember

Point 1

Window functions use:

```
OVER()
```

Point 2

`PARTITION BY` creates groups without removing rows.

Point 3

`ORDER BY` inside `OVER()` controls the order used by the window function.

Point 4

```
LAG  → Previous
LEAD → Next
```

Point 5

```
ROW_NUMBER → Always unique numbering
RANK       → Same rank + gaps
DENSE_RANK → Same rank + no gaps
```

Point 6

Window functions are useful for:

- Ranking

- Running totals
- Comparing previous/next rows
- Department-wise calculations
- Finding top/bottom records
- Moving calculations
- Comparing individual values with group averages

One-Line Definition for Exam

> A window function performs a calculation across a set of related rows while preserving the individual rows in the result.

Most important syntax to memorize

```
function() OVER(
    PARTITION BY column
    ORDER BY column
)
```

And remember:

```
PARTITION BY = Divide into groups

ORDER BY     = Decide the order

OVER()       = Make it a window calculation
```

-